MA3632 Lecture 7 – Logistic Regression and Classification

Lectures 5 and 6 developed the ERM framework mainly in the regression setting, where the output space $\mathcal{Y} \subseteq \mathbb{R}$ is continuous, though Lecture 5 also introduced k-nearest neighbours as an instance-based approach to classification. This lecture turns to *model-based classification*, where $\mathcal{Y}$ is a finite set of class labels. The transition requires two changes: the loss function must be adapted to discrete outputs, and the hypothesis class must produce class predictions, class probabilities, or scores from which class predictions can be obtained.

Some of the material here – the sigmoid function, the decision boundary, the basic fitting procedure – may be familiar from the companion module MA3622. The aim of this lecture is not to repeat that account but to place logistic regression firmly within the ERM framework developed in Lectures 5 and 6, to derive the cross-entropy loss from maximum likelihood principles, and to extend the binary case to multiclass problems via the softmax function. The final section introduces classification performance metrics, which will be needed again in the model evaluation discussion of Week 10.

1 From Regression to Classification

In binary classification, $\mathcal{Y} = \{0, 1\}$. A natural first attempt is to apply OLS directly: fit a linear model $\hat{f}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ and threshold the output at $1/2$. This approach has two deficiencies. First, the linear model produces values in $\mathbb{R}$, and predictions far from the decision boundary are penalised by the squared loss even when they are correctly and confidently classified. Second, the predictions cannot be directly interpreted as probabilities, which are often required downstream (for calibration, cost-sensitive decisions, or as inputs to a downstream model).

The resolution is to model the *conditional probability* $P(y = 1 \mid \mathbf{x})$ rather than y itself, and to use a function that maps $\mathbb{R}$ to $(0, 1)$ to ensure valid probability outputs.

Definition 1.1. The **sigmoid function** (also called the logistic function) is

$$\sigma(t) \;=\; \frac{1}{1 + e^{-t}}, \qquad t \in \mathbb{R}.$$

It satisfies $\sigma(t) \in (0, 1)$ for all t, $\sigma(0) = 1/2$, $\sigma(-t) = 1 - \sigma(t)$, and $\sigma'(t) = \sigma(t)(1 - \sigma(t))$.

The **logistic regression model** specifies

$$P(y = 1 \mid \mathbf{x};\, \mathbf{w}) \;=\; \sigma(\mathbf{w}^\top \mathbf{x}) \;=\; \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})},$$

where $\mathbf{x} \in \mathbb{R}^{p+1}$ includes an intercept term and $\mathbf{w} \in \mathbb{R}^{p+1}$ is the parameter vector. The predicted class is

$$\hat{y} \;=\; \mathbf{1}\Big[\sigma(\mathbf{w}^\top \mathbf{x}) \geq \tfrac{1}{2}\Big] \;=\; \mathbf{1}\Big[\mathbf{w}^\top \mathbf{x} \geq 0\Big].$$

Remark 1.2. The name *logistic regression* is historical: despite the word regression, the model is used for classification. The term arises because the model is linear in the *log-odds* (logit): if $p = P(y = 1 \mid \mathbf{x})$, then $\log(p/(1 - p)) = \mathbf{w}^\top \mathbf{x}$. This is a regression of the log-odds on the features, and it is this quantity – not the probability – that is modelled as linear.

1.1 The decision boundary

The **decision boundary** of the logistic classifier is the set $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} = 0\}$, a hyperplane in $\mathbb{R}^p$. Points on one side are assigned class 1 and points on the other are assigned class 0. The logistic model therefore defines a *linear classifier*: its decision boundary is always a hyperplane, regardless of the feature values.

To model nonlinear decision boundaries, one can apply the same feature map ideas as in polynomial regression: replace $\mathbf{x}$ with $\boldsymbol{\phi}(\mathbf{x})$ (e.g. polynomial or interaction features) and fit logistic regression in the expanded feature space. The boundary is nonlinear in the original features but linear in $\boldsymbol{\phi}(\mathbf{x})$.

2 Maximum Likelihood Estimation and the Cross-Entropy Loss

We now derive the loss function for logistic regression from first principles. Under the model, each observation $(y_i, \mathbf{x}_i)$ contributes a Bernoulli likelihood:

$$P(y_i \mid \mathbf{x}_i;\, \mathbf{w}) \;=\; \sigma(\mathbf{w}^\top \mathbf{x}_i)^{y_i} \big(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i)\big)^{1-y_i}.$$

Under the i.i.d. assumption, the joint likelihood of the training set is

$$L(\mathbf{w}) \;=\; \prod_{i=1}^{n} P(y_i \mid \mathbf{x}_i;\, \mathbf{w}).$$

Taking logarithms, the log-likelihood is

$$\ell(\mathbf{w}) \;=\; \sum_{i=1}^{n} \Big[y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) \;+\; (1 - y_i) \log\big(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i)\big) \Big].$$

Maximising $\ell(\mathbf{w})$ is equivalent to minimising $-\ell(\mathbf{w})/n$.

Definition 2.1. The **binary cross-entropy loss** (also called the *log loss*) for a single observation is

$$\ell_{\mathrm{CE}}(y, \hat{p}) \;=\; -\big[y \log \hat{p} + (1 - y) \log(1 - \hat{p}) \big],$$

where $\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x}) \in (0, 1)$ is the predicted probability of class 1. The empirical risk under cross-entropy loss is

$$\hat{R}(\mathbf{w}) \;=\; \frac{1}{n} \sum_{i=1}^{n} \ell_{\mathrm{CE}}(y_i,\, \sigma(\mathbf{w}^\top \mathbf{x}_i)).$$

Minimising $\hat{R}(\mathbf{w})$ is therefore exactly maximum likelihood estimation of $\mathbf{w}$ under the logistic model. This is the connection between the ERM framework and the probabilistic (MLE) perspective: they produce the same estimator, with the loss function playing the role of the negative log-likelihood.

Proposition 2.2. *The empirical risk $\hat{R}(\mathbf{w})$ under cross-entropy loss is a convex function of $\mathbf{w}$. Unlike OLS, logistic regression does not in general admit a closed-form solution; its gradient is*

$$\nabla_{\mathbf{w}} \hat{R}(\mathbf{w}) \;=\; \frac{1}{n} \sum_{i=1}^{n} \big(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i\big) \mathbf{x}_i \;=\; \frac{1}{n} X^\top (\hat{\mathbf{p}} - \mathbf{y}),$$

where $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$ and X is the design matrix.

Proof. Differentiating $\hat{R}(\mathbf{w})$ with respect to $\mathbf{w}$ and using $\sigma'(t) = \sigma(t)(1-\sigma(t))$:

$$\begin{aligned}\frac{\partial \hat{R}}{\partial \mathbf{w}} &= -\frac{1}{n}\sum_{i=1}^{n}\left[\frac{y_i}{\hat{p}_i} - \frac{1-y_i}{1-\hat{p}_i}\right]\hat{p}_i(1-\hat{p}_i)\,\mathbf{x}_i \\ &= -\frac{1}{n}\sum_{i=1}^{n}\big[y_i(1-\hat{p}_i) - (1-y_i)\hat{p}_i\big]\mathbf{x}_i \;=\; \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)\mathbf{x}_i.\end{aligned}$$

Convexity follows because $\hat{R}$ is a sum of compositions of convex functions with affine maps, and the Hessian $n^{-1}X^\top \mathrm{diag}(\hat{p}_i(1-\hat{p}_i))X$ is positive semidefinite. □

Remark 2.3. The gradient has a striking structural resemblance to the OLS gradient: it is a weighted sum of the feature vectors $\mathbf{x}_i$, with weights equal to the *residuals* $\hat{p}_i - y_i$. In OLS the residuals are $\hat{y}_i - y_i$; here they are $\hat{p}_i - y_i$ with predicted probabilities in place of predicted values. This is not a coincidence, though it is also not an instance of a fully general ERM gradient formula: it reflects the particular structure of squared-error linear regression and cross-entropy logistic regression, in both of which the gradient of the empirical risk can be expressed as the design matrix multiplied by a vector of prediction residuals. This convenient form does not hold for arbitrary loss functions or hypothesis classes.

3 Gradient Descent

Since $\hat{R}(\mathbf{w})$ does not in general admit a closed-form solution, we use an iterative algorithm. Gradient descent is the simplest such algorithm and underlies most of the optimisation methods used in machine learning.

Definition 3.1. Gradient descent for minimising a differentiable function $f(\mathbf{w})$ initialises $\mathbf{w}^{(0)}$ (e.g. at zero) and iterates

$$\mathbf{w}^{(t+1)} \;=\; \mathbf{w}^{(t)} - \eta\,\nabla f(\mathbf{w}^{(t)}), \qquad t = 0, 1, 2, \ldots,$$

where $\eta > 0$ is the **learning rate** (also called the step size).

For logistic regression, each iteration moves $\mathbf{w}$ in the direction that most rapidly decreases $\hat{R}$, scaled by η. When a finite minimiser exists, convexity of $\hat{R}$ ensures that gradient descent with a suitable learning rate converges to a global minimiser.

Remark 3.2. This qualification is not vacuous for logistic regression. If the training data are *linearly separable* – if some hyperplane perfectly separates the two classes – then $\hat{R}(\mathbf{w})$ has no finite minimiser: scaling any separating $\mathbf{w}$ by an increasingly large positive constant drives every predicted probability towards its correct label and $\hat{R}$ towards its infimum of zero, but that infimum is never attained at finite $\|\mathbf{w}\|$. In this *complete separation* case, gradient descent does not converge to a finite point; the iterates grow without bound and the fitted coefficients become arbitrarily large (a symptom visible in practice as failure to converge, or absurdly large coefficient magnitudes). Regularisation, introduced in Section 4, prevents the coefficient divergence associated with separation and typically yields a finite fitted solution.

Proposition 3.3. *If $\hat{R}$ is convex, has a finite minimiser $\mathbf{w}^*$, and has L-Lipschitz-continuous gradient (i.e. $\|\nabla\hat{R}(\mathbf{u}) - \nabla\hat{R}(\mathbf{v})\| \le L\|\mathbf{u}-\mathbf{v}\|$ for all $\mathbf{u}, \mathbf{v}$), then gradient descent with step size $\eta \le 1/L$ satisfies*

$$\hat{R}(\mathbf{w}^{(T)}) - \hat{R}(\mathbf{w}^*) \;\le\; \frac{\|\mathbf{w}^{(0)} - \mathbf{w}^*\|^2}{2\eta T}.$$

The convergence rate is $O(1/T)$.

For logistic regression, since $\hat{p}_i(1-\hat{p}_i) \le 1/4$, the gradient is L-Lipschitz for $L = \|X^\top X\|_{\mathrm{op}}/(4n)$ – that is, this value of L satisfies the Lipschitz condition, though it need not be the smallest such constant – so the step size $\eta = 4n/\|X^\top X\|_{\mathrm{op}}$ is safe whenever a finite minimiser exists. In practice, η is tuned by trial or by line search.

3.1 Newton's method

Newton's method uses second-order information to converge faster than gradient descent. The update is

$$\mathbf{w}^{(t+1)} \;=\; \mathbf{w}^{(t)} - \left[\nabla^2 \hat{R}(\mathbf{w}^{(t)})\right]^{-1} \nabla \hat{R}(\mathbf{w}^{(t)}),$$

where $\nabla^2 \hat{R}(\mathbf{w}) = n^{-1} X^\top W X$ and $W = \mathrm{diag}(\hat{p}_i(1-\hat{p}_i))$. For logistic regression this leads to *iteratively reweighted least squares* (IRLS): each Newton step can be expressed as a weighted least-squares problem for a working response, with weights $\hat{p}_i(1-\hat{p}_i)$. Under standard regularity conditions, Newton's method converges quadratically near the optimum but requires inverting an $(p+1)\times(p+1)$ matrix at each step, which is expensive when p is large. In practice, limited-memory quasi-Newton methods (L-BFGS) approximate the Hessian and are the default solver in most software.

4 Regularisation

Exactly as in ridge and Lasso regression, logistic regression can overfit when p is large relative to n or when predictors are nearly collinear; as noted in the remark above, regularisation also addresses the coefficient divergence that occurs on linearly separable data. The same penalties apply, and as in Lecture 6 the intercept is excluded from the penalty.

Definition 4.1. Write the model as $f(\mathbf{x}) = w_0 + \mathbf{x}^\top \mathbf{w}$, where $w_0 \in \mathbb{R}$ is the intercept and $\mathbf{w} = (w_1, \dots, w_p)^\top \in \mathbb{R}^p$ are the remaining coefficients. **Regularised logistic regression** minimises

$$\hat{R}_\lambda(w_0, \mathbf{w}) \;=\; \frac{1}{n}\sum_{i=1}^{n} \ell_{\mathrm{CE}}(y_i,\, \sigma(w_0 + \mathbf{x}_i^\top \mathbf{w})) \;+\; \lambda \|\mathbf{w}\|_q^q,$$

where $q = 2$ gives ℓ_2 (Ridge) regularisation and $q = 1$ gives ℓ_1 (Lasso) regularisation, and the penalty applies only to $\mathbf{w}$, not to w_0.

Remark 4.2. In scikit-learn's `LogisticRegression`, regularisation strength is controlled by the parameter `C`: smaller `C` corresponds to stronger regularisation and larger `C` to weaker regularisation (close to unpenalised MLE when a finite MLE exists). The precise relationship between `C` and a λ defined as above depends on the exact scaling convention used for the objective, so `C` should not be read as being literally $1/\lambda$ under every parameterisation; what matters for practice is the direction of the relationship. This convention – smaller `C` meaning more regularisation – is the opposite of the one used for λ in ridge regression in Week 6, and it originates in the SVM literature. When tuning logistic regression by cross-validation, one searches over a grid of `C` values and selects the one that minimises cross-validation error.

5 Multiclass Classification

When $\mathcal{Y} = \{1, \dots, C\}$ with $C > 2$, two approaches are common.

5.1 One-vs-rest

In the **one-vs-rest** (OvR) scheme, C binary classifiers are trained, one for each class. Classifier c is trained to distinguish class c from all other classes, with the training set relabelled so that class c is the positive class and all other classes are negative. At prediction time, each classifier produces a score $\hat{p}_c(\mathbf{x})$, and the predicted class is the one with the highest score:

$$\hat{y} \;=\; \arg\max_{c\in\{1,\dots,C\}} \hat{p}_c(\mathbf{x}).$$

OvR is simple and works well when the classes are well-separated, but the scores $\hat{p}_c(\mathbf{x})$ are not calibrated to sum to one, so they cannot be directly interpreted as probabilities.

5.2 Softmax regression

Softmax regression (also called multinomial logistic regression) is a single model that directly parameterises the probability of each class. Let $\mathbf{w}_c \in \mathbb{R}^{p+1}$ be the parameter vector for class c. The model specifies

$$P(y = c \mid \mathbf{x};\, W) \;=\; \frac{\exp(\mathbf{w}_c^\top \mathbf{x})}{\sum_{j=1}^{C} \exp(\mathbf{w}_j^\top \mathbf{x})} \;=:\; \mathrm{softmax}(\mathbf{W}\mathbf{x})_c,$$

where $W = (\mathbf{w}_1, \dots, \mathbf{w}_C)^\top \in \mathbb{R}^{C\times(p+1)}$. The probabilities are positive and sum to one by construction.

The softmax generalises the sigmoid: when $C = 2$, the softmax probability for class 1 reduces to $\sigma(\mathbf{w}^\top \mathbf{x})$ where $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_2$.

Definition 5.1. The **categorical cross-entropy loss** for a single observation with true class y and predicted probabilities $\hat{\mathbf{p}} = (\hat{p}_1, \dots, \hat{p}_C)$ is

$$\ell_{\mathrm{CE}}(y, \hat{\mathbf{p}}) \;=\; -\log \hat{p}_y \;=\; -\log P(y \mid \mathbf{x};\, W).$$

This is the negative log-likelihood of the true class under the predicted distribution. A perfect prediction ($\hat{p}_y = 1$) gives loss zero; a prediction of near-zero probability for the true class gives large loss. Minimising the empirical risk under categorical cross-entropy is again equivalent to MLE of the softmax parameters.

Remark 5.2. The softmax function *is* invariant to adding a constant to all scores: adding c to every $\mathbf{w}_j^\top \mathbf{x}$ cancels in the ratio and leaves the probabilities unchanged. Because of this invariance the parameterisation W is not identifiable as stated – infinitely many choices of W give the same probabilities – so one typically imposes the constraint $\mathbf{w}_C = \mathbf{0}$ (reference class) to fix this redundancy, reducing the effective number of parameters from $C(p+1)$ to $(C-1)(p+1)$.

6 Classification Performance Metrics

Once a classifier is fitted, evaluating it requires metrics that go beyond the raw accuracy rate. Accuracy is misleading whenever the class distribution is imbalanced: a classifier that always predicts the majority class can achieve high accuracy while being useless.

For binary classification, the outcomes of predictions on a test set can be organised into a 2×2 **confusion matrix**:

	Predicted positive	Predicted negative
Actual positive	True positive (TP)	False negative (FN)
Actual negative	False positive (FP)	True negative (TN)

Definition 6.1. From the confusion matrix entries TP, FP, FN, TN, the standard derived metrics are:

(i) **Accuracy**: $(TP + TN)/(TP + FP + FN + TN)$.

(ii) **Precision**: $TP/(TP + FP)$ – of all positive predictions, what fraction are correct?

(iii) **Recall** (sensitivity): $TP/(TP + FN)$ – of all actual positives, what fraction are detected?

(iv) **Specificity**: $TN/(TN + FP)$ – of all actual negatives, what fraction are correctly rejected?

(v) **F_1 score**: the harmonic mean of precision and recall, $F_1 = 2 \cdot \text{Precision} \cdot \text{Recall}/(\text{Precision} + \text{Recall})$.

Precision and recall capture different failure modes and are often in tension. A classifier that predicts nearly every observation as positive may achieve very high recall – few actual positives are missed – but poor precision, since many of its positive predictions will be wrong. Conversely, a classifier that predicts positive only in the most certain cases may achieve high precision but poor recall, missing many true positives it was not confident enough to flag. The F_1 score penalises classifiers that sacrifice one for the other.

The choice of which metric to prioritise depends on the relative cost of the two error types. In medical screening, a false negative (missed disease) may be far more costly than a false positive (unnecessary follow-up), so recall is prioritised. In fraud detection, a false positive (legitimate transaction blocked) may be costly in customer experience terms, so precision may matter more.

6.1 The ROC curve and AUC

Many classifiers output a continuous score $s(\mathbf{x})$ (for example, a predicted probability $\hat{p}$) and produce a binary prediction by thresholding this score at some value τ: predict class 1 if $s(\mathbf{x}) \geq \tau$. Different values of τ give different trade-offs between the true positive rate (recall) and the false positive rate ($FP/(FP + TN)$).

Definition 6.2. The **receiver operating characteristic (ROC) curve** is the plot of the true positive rate against the false positive rate as τ varies from 1 to 0. The **area under the ROC curve** (AUC) is

$$\text{AUC} = \int_0^1 \text{TPR}(FPR)\, d(FPR).$$

An AUC of 1 indicates a perfect classifier. An AUC of 1/2 indicates a classifier with no ability to discriminate between the classes – equivalent to random ranking of observations – while an AUC below 1/2 indicates that the classifier's scores are systematically ranked in the wrong direction (in which case simply reversing the ranking would improve performance).

AUC is a threshold-free metric: it summarises classifier performance across all possible decision thresholds, making it useful when the optimal threshold is not known in advance or varies across deployment contexts. It also has a probabilistic interpretation: the AUC equals the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative example.

Remark 6.3. For multiclass problems, the confusion matrix generalises to a $C \times C$ matrix. Macro-averaged metrics compute the binary metric for each class and average, giving equal weight to every class regardless of its size; micro-averaged metrics aggregate TP, FP, FN across all classes before computing a single value, so classes with more observations contribute more to the result. When classes are imbalanced, macro-averaging can therefore reveal poor minority-class performance that would otherwise be obscured by a micro-averaged metric dominated by the majority class.

7 Exercises

Exercise 7.1. Show that the logistic regression model can be written as a linear model in the log-odds. That is, let $p(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$ and show that

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \;=\; \mathbf{w}^\top \mathbf{x}.$$

Hence explain why the decision boundary $\{p(\mathbf{x}) = 1/2\}$ is a hyperplane in $\mathbb{R}^p$. For a two-dimensional feature vector $(x_1, x_2)^\top$ with parameters $\mathbf{w} = (w_0, w_1, w_2)^\top$, write down the equation of the decision boundary explicitly and describe its shape.

Solution. By definition, $p(\mathbf{x}) = (1+e^{-\mathbf{w}^\top \mathbf{x}})^{-1}$, so $1-p(\mathbf{x}) = e^{-\mathbf{w}^\top \mathbf{x}}(1+e^{-\mathbf{w}^\top \mathbf{x}})^{-1}$. Writing $z = \mathbf{w}^\top \mathbf{x}$ for brevity,

$$\frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \;=\; \frac{1/(1+e^{-z})}{e^{-z}/(1+e^{-z})} \;=\; \frac{1}{e^{-z}} \;=\; e^{z}.$$

Taking logarithms gives $\log(p/(1-p)) = z = \mathbf{w}^\top \mathbf{x}$.
The decision boundary is where $p(\mathbf{x}) = 1/2$, i.e. where the log-odds equal zero: $\mathbf{w}^\top \mathbf{x} = 0$. This is a linear equation in $\mathbf{x}$, so it defines a hyperplane in $\mathbb{R}^p$.
In two dimensions with intercept, $\mathbf{x} = (1, x_1, x_2)^\top$ and the decision boundary is $w_0 + w_1 x_1 + w_2 x_2 = 0$, or equivalently $x_2 = -(w_0 + w_1 x_1)/w_2$ (assuming $w_2 \neq 0$). This is a straight line in the (x_1, x_2) plane. □

Exercise 7.2. A logistic regression model is fitted on a training set of 100 observations (50 positive, 50 negative). On a separate test set of 200 observations (40 positive, 160 negative), the model produces the following confusion matrix:

	Predicted positive	Predicted negative
Actual positive	30	10
Actual negative	20	140

Compute the accuracy, precision, recall, specificity, and F_1 score. A colleague suggests that accuracy alone adequately summarises model performance here. Assess this claim, paying attention to the class distribution in the test set.

Solution. From the confusion matrix: $TP = 30$, $FN = 10$, $FP = 20$, $TN = 140$.

$$\begin{aligned}
\text{Accuracy} &= (30 + 140)/200 = 170/200 = 0.85,\\
\text{Precision} &= 30/(30 + 20) = 30/50 = 0.60,\\
\text{Recall} &= 30/(30 + 10) = 30/40 = 0.75,\\
\text{Specificity} &= 140/(140 + 20) = 140/160 = 0.875,\\
F_1 &= 2 \times 0.60 \times 0.75/(0.60 + 0.75) = 0.90/1.35 \approx 0.667.
\end{aligned}$$

Accuracy of 0.85 sounds impressive, but consider the null classifier that always predicts the majority class (negative): it would achieve accuracy $160/200 = 0.80$, only slightly below. The model achieves this by correctly classifying nearly all the majority-class examples (specificity 0.875) while missing 25% of the positive examples (recall 0.75). If the positive class represents, say, a disease or a fraud event, missing 10 out of 40 cases is a significant failure that accuracy obscures. Precision and recall – summarised in the F_1 score – give a more complete picture, particularly when the class distribution is as skewed (4:1 negative-to-positive) as it is here. □

Exercise 7.3. Let $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$. Starting from the cross-entropy empirical risk

$$\hat{R}(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\big[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\big],$$

show that one step of gradient descent with learning rate η updates $\mathbf{w}$ as

$$\mathbf{w} \;\leftarrow\; \mathbf{w} \;-\; \frac{\eta}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)\mathbf{x}_i.$$

Verify this update rule for the case $n = 1$, $\mathbf{x} = (1, 2)^\top$, $y = 1$, $\mathbf{w} = (0, 0)^\top$, and $\eta = 0.5$.

Solution. The gradient was derived in Proposition 2.2: $\nabla_{\mathbf{w}} \hat{R}(\mathbf{w}) = n^{-1}\sum_{i=1}^{n}(\hat{p}_i - y_i)\mathbf{x}_i$. Substituting into the gradient descent update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \hat{R}$ gives the stated rule.
For the numerical case: $\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x}) = \sigma(0 \cdot 1 + 0 \cdot 2) = \sigma(0) = 0.5$. The gradient is $(\hat{p} - y)\mathbf{x} = (0.5 - 1)(1, 2)^\top = (-0.5, -1)^\top$. The update is

$$\mathbf{w} \;\leftarrow\; (0, 0)^\top - 0.5 \times (-0.5, -1)^\top = (0.25, 0.5)^\top.$$

After one step, the model predicts $\sigma(0.25 \times 1 + 0.5 \times 2) = \sigma(1.25) \approx 0.778$, which is higher than the initial 0.5 and moving in the correct direction towards the true label $y = 1$. □

Further Reading

Lindholm, A., Wahlström, N., Lindsten, F., and Schön, T. B. (2022). *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press. Treats logistic regression and gradient-based fitting as a direct parametric counterpart to the regression models of Week 6, at a level consistent with this lecture.

James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Chapter 4 covers logistic regression, the multinomial extension, and classification performance metrics including the confusion matrix and ROC curve used in Sections 6 and 6.1.

Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society, Series B*, 20(2), 215–232. An early and influential paper establishing logistic regression for binary outcome data.

References

Cox, D.R., 1958. The regression analysis of binary sequences. *Journal of the Royal Statistical Society Series B: Statistical Methodology, 20*(2), pp.215–232.

James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J., 2023. *An Introduction to Statistical Learning with Applications in Python*. Springer.

Lindholm, A., Wahlström, N., Lindsten, F. and Schön, T.B., 2022. *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press.